

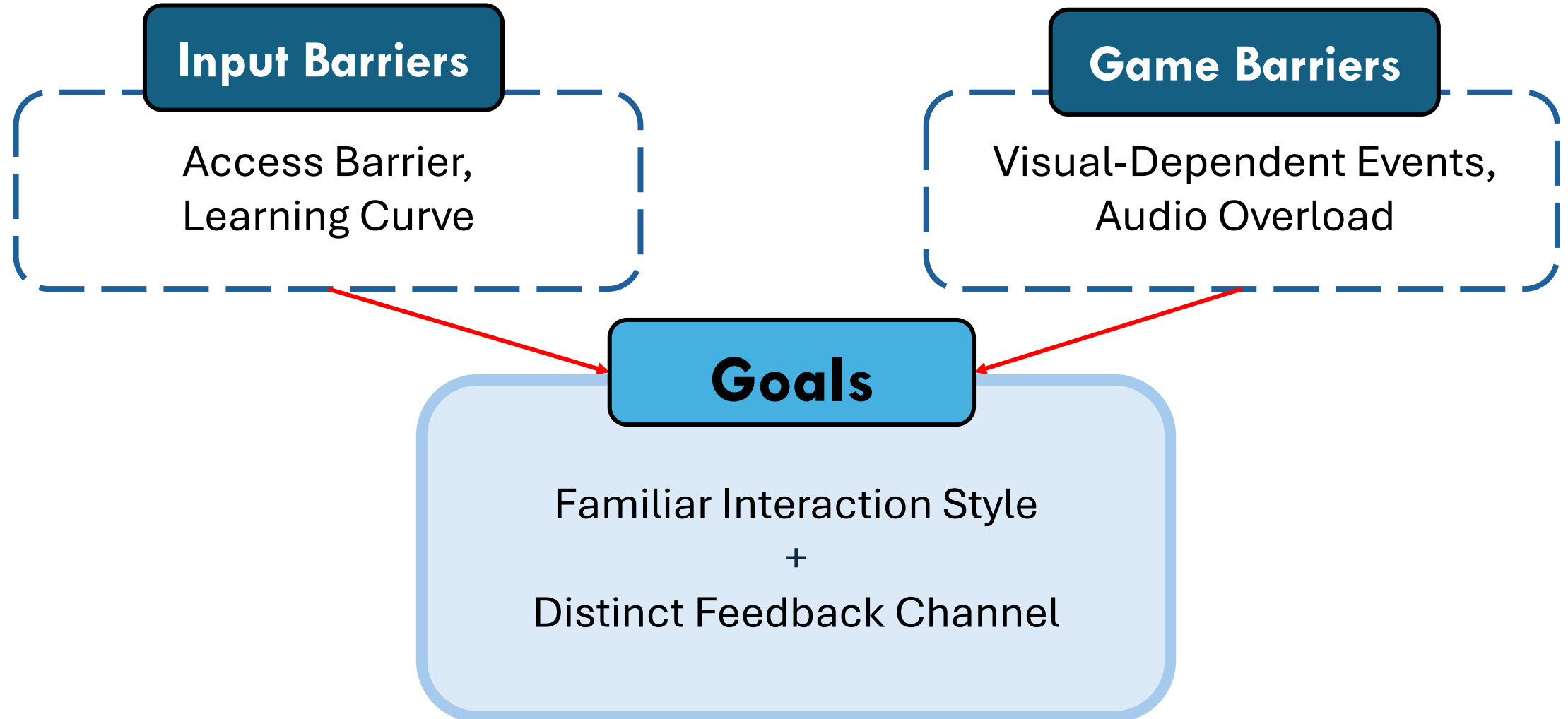
Accessible Smartphone Controller for Unity

Colleen Cipriano

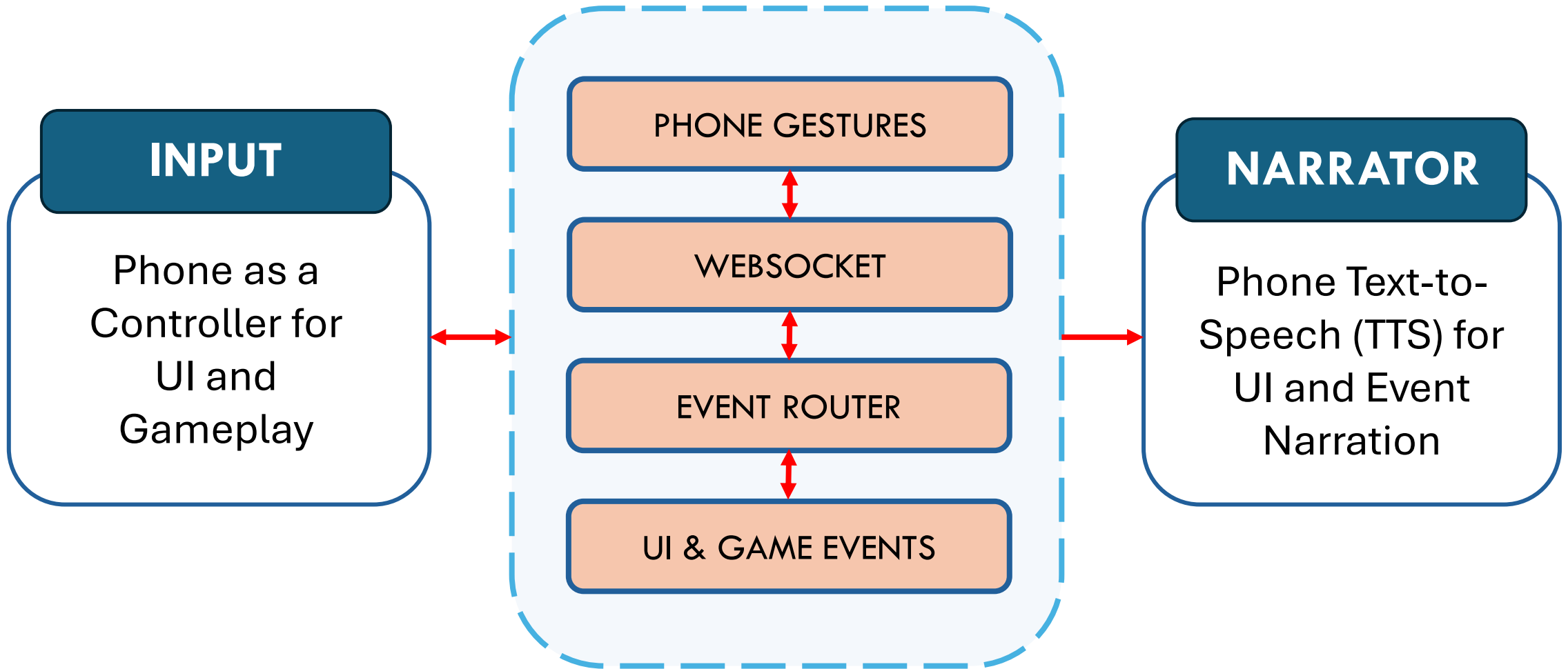
Who is ASCU for?

ASCU is for **blind and low-vision (BLV)** players to use a device they already know as both the **controller** and a separate **narration channel** for game events.

Problem Space



System Overview



Unity Core Components

Phone Gestures

MobileInput/
MobileNetworking/

AccessibilityEvents
Router/

FeedbackManager/

- ✓ MobileInput
 - MobileInputListener.cs
- ✓ MobileNetworking
 - MobileInputMessage.cs
 - WebSocketConnect.cs
 - WebSocketTest.cs
- ✓ EventFeedback
 - AccessibilityEvents.cs
 - AccessibilityEventsRouter.cs
 - DebugFeedbackPlug.cs
 - EventFeedbackSettings.cs
 - FeedbackManager.cs
 - FeedbackPayload.cs
 - FeedbackPlug.cs

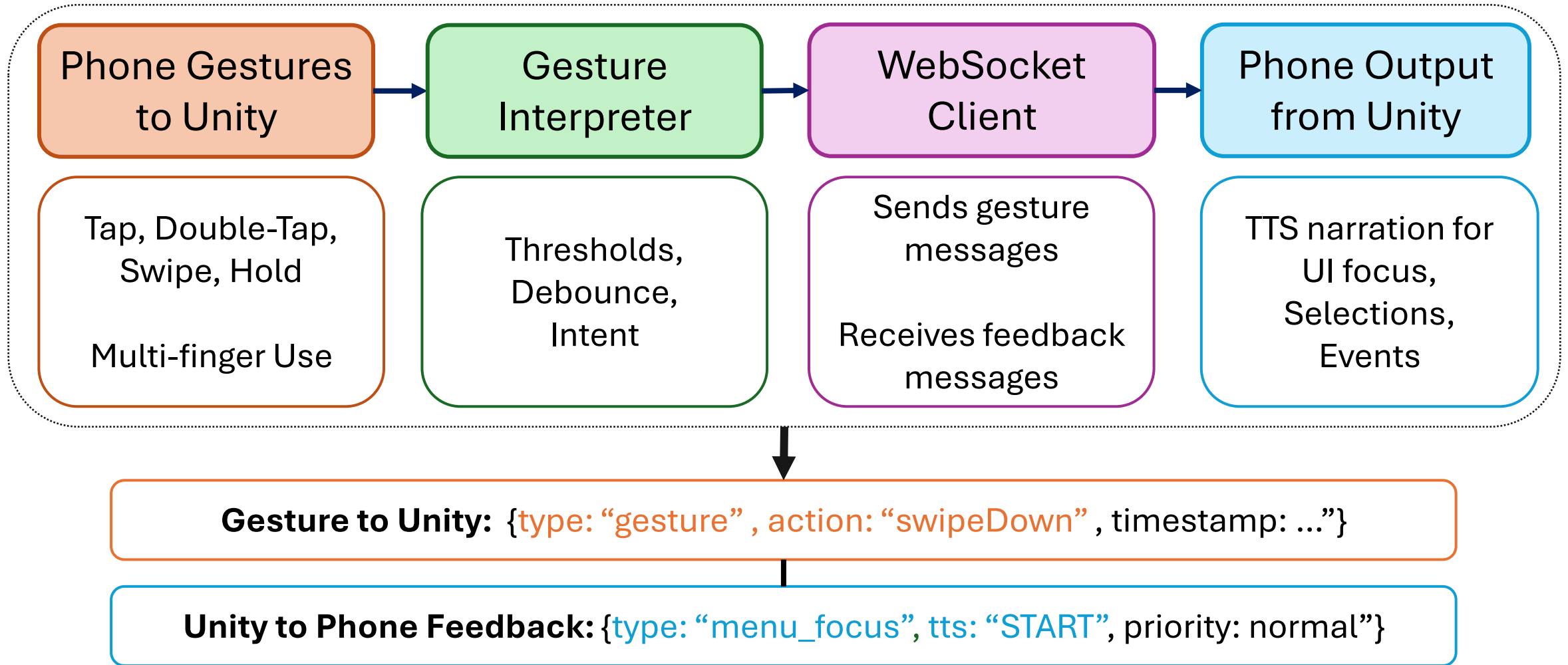


Demo Scenes

MenuUI/
DemoGameplay/

- ✓ MenuUI
 - InstructionsScene.cs
 - MainMenuManager.cs
 - MobileConnectInstructions.cs
- ✓ DemoGameplay
 - ExplorationController.cs
 - ExplorationEventTrigger.cs
 - GameSceneBootstrap.cs
 - HazardAnnouncer.cs
 - HazardProximity.cs
 - PlayerAudio.cs
 - PlayerCollision.cs
 - PlayerMovement.cs

Phone Controller Components



DEMO SCENES

```
JS controller.js > getWebSocketURL
22 function addDebugLog(msg) {
23     console.log(msg);
24 }
25
26 // Initialize orientation on page load
27 window.addEventListener('load', updateOrientationHint);
28
29 // ----- INPUT (PHONE -> UNITY) ----- //
30 /**
31  * If HARCODED_WS_URL is non-empty, always use that for a tunnel.
32  * Otherwise, connect back to the same host that served this page on port 8081.
33  */
34
35 const HARCODED_WS_URL = "";
36
37 function getWebSocketURL() {
38     if (HARCODED_WS_URL) {
39         return HARCODED_WS_URL;
40     }
41
42     const protocol = location.protocol === "https:" ? "wss://" : "ws://";
43     return protocol + location.hostname + ":8081";
44 }
45
46 // Resolved WebSocket URL used by connectToUnity()
47 const WS_URL = getWebSocketURL();
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
WebSocketConnect.cs
using UnityEngine;
using System;
using WebSocketSharp;
using WebSocketSharp.Server;

/// <summary>
/// WebSocket server that connects Unity to the phone controller web page.
/// Uses WebSocketSharp's WebSocketServer (https://github.com/sta/websocket-sharp)
/// and implements FeedbackPlug so FeedbackManager can send JSON feedback,
/// while raising OnClientMessage when input messages arrive from the phone.
/// </summary>

public class WebSocketConnect : MonoBehaviour, FeedbackPlug
{
    [Header("Server Settings")]
    [Tooltip("TCP port to listen on for phone controller connections (ws://0.0.0.0:port/).")]
    public int port = 8081;

    /// <summary>
    /// Fired on the Unity main thread when a JSON message is received
    /// from the phone controller.
    /// </summary>
    public static event Action<string> OnClientMessage;

    // WebSocketSharp server instance
    private WebSocketServer _server;

    // Thread-safe queue for messages coming from WebSocketSharp's background thread.
    private static readonly ConcurrentQueue<string> _messageQueue = new ConcurrentQueue<string>();
}
```

```
colleencipriano@MacBook-Pro Phone-Controller % python3 -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

Phone URL: <laptop-ip>:8000/controller.html

MAIN MENU SCENE DEMO

Unity Version Control

Console

Clear Collapse 4 0 0

[15:35:29] [WebSocketTransport] Server started on ws://
UnityEngine.Debug.Log (object)

[15:35:29] [BLV.JSON] ("eventId": "menu_scene_loaded",
UnityEngine.Debug.Log (object)

[15:35:29] [WebSocketTransport] Phone connected.
UnityEngine.Debug.Log (object)

[15:35:32] [WebSocketTransport] Phone disconnected.
UnityEngine.Debug.Log (object)

Game

Display 1 Free Aspect Scale 1x Play Focused Stats Giz

Hierarchy

MainMenuDemo

- Main Camera
- StartMenuCanvas
- EventSystem
- MainMenuManager
- MobileControllerSystem

Inspector

Project

All Materials
All Models
All Prefabs

Assets

- MobileControllerSystem
 - AccessibilityFeedbackProfiles
 - Audio
 - Documentation
 - PhysicsMaterial
 - Prefab
 - Scenes
 - Scripts
 - DemoGameplay

Assets > MobileControllerSystem > AccessibilityFeedbackProfiles

DefaultSpeech MenuActivateFe... MenuFocusFeed... MenuSceneLoad...

[WebSocketTransport] Phone disconnected.

EXPLORATION SCENE DEMO

CC Unity Version Control



Layers Layout

Console

Clear Collapse

1 0 0

Scene

Game

Game Display 1 Free Aspect

Scale

1x

Play Focused



Stats

Gizmos

Hierarchy

Inspector

Open

Main Menu Demo (Scene As: ...)

[16:07:20] [WebSocketTransport] Server
UnityEngine.Debug.Log (object)

[16:07:20] [BLV JSON] ["eventId": "menu",
UnityEngine.Debug.Log (object)

[16:07:21] [WebSocketTransport] Phone
UnityEngine.Debug.Log (object)

START

INSTRUCTIONS

CONNECTION
HELP

Project

+

Q: All Materials

Q: All Models

Q: All Prefabs

Assets

MobileControllerSystem

AccessibilityFeedbackProfiles

Audio

Documentation

PhysicsMaterial

Prefab

Scenes

Scripts

DemoGameplay

Assets > MobileControllerSystem > Scenes



ExplorationDemo



MainMenuDemo

Assets/MobileControllerSystem/Scenes/MainMenuDemo.unity

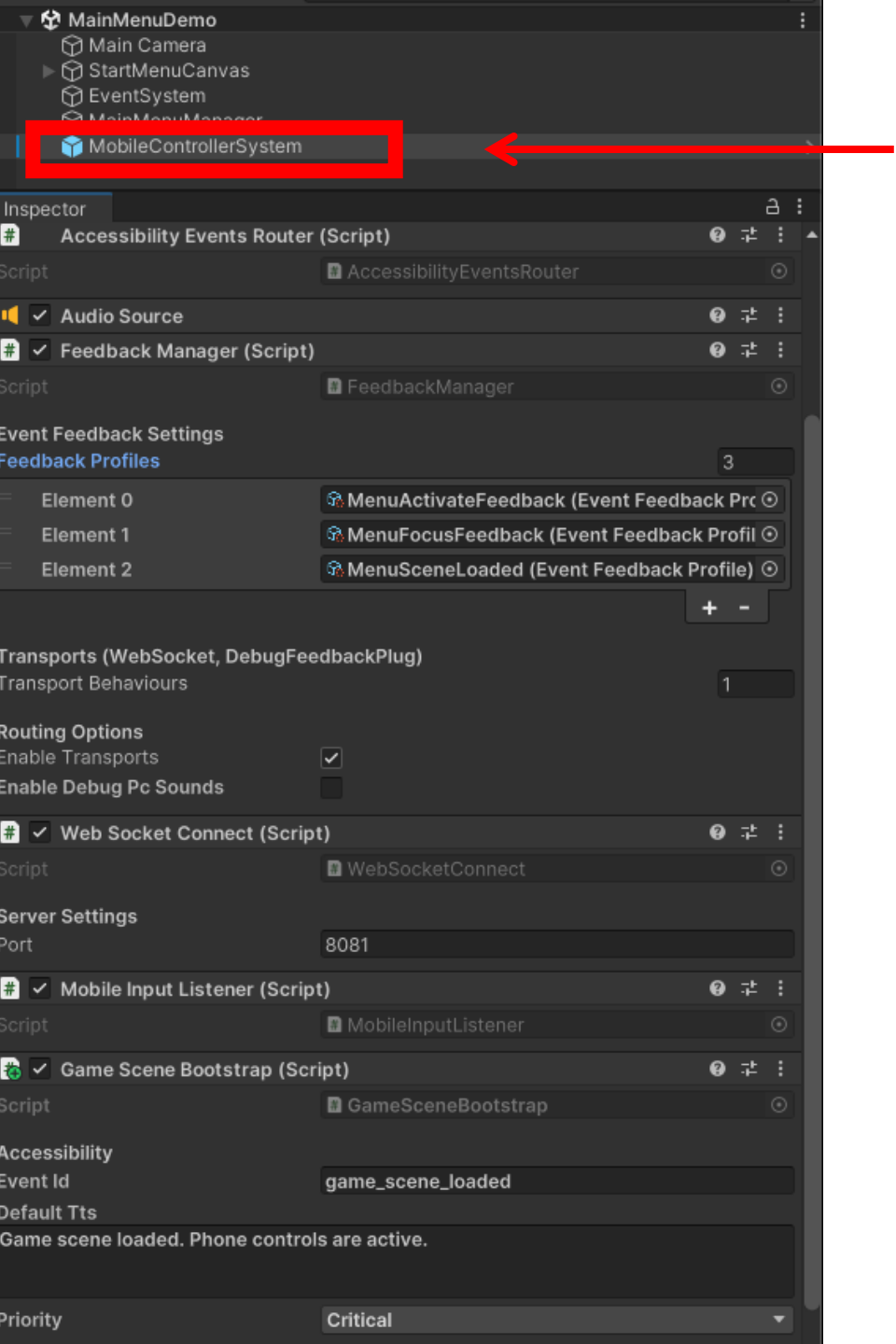
[WebSocketTransport] Phone connected.

Asset Labels

AssetBundle

None

None

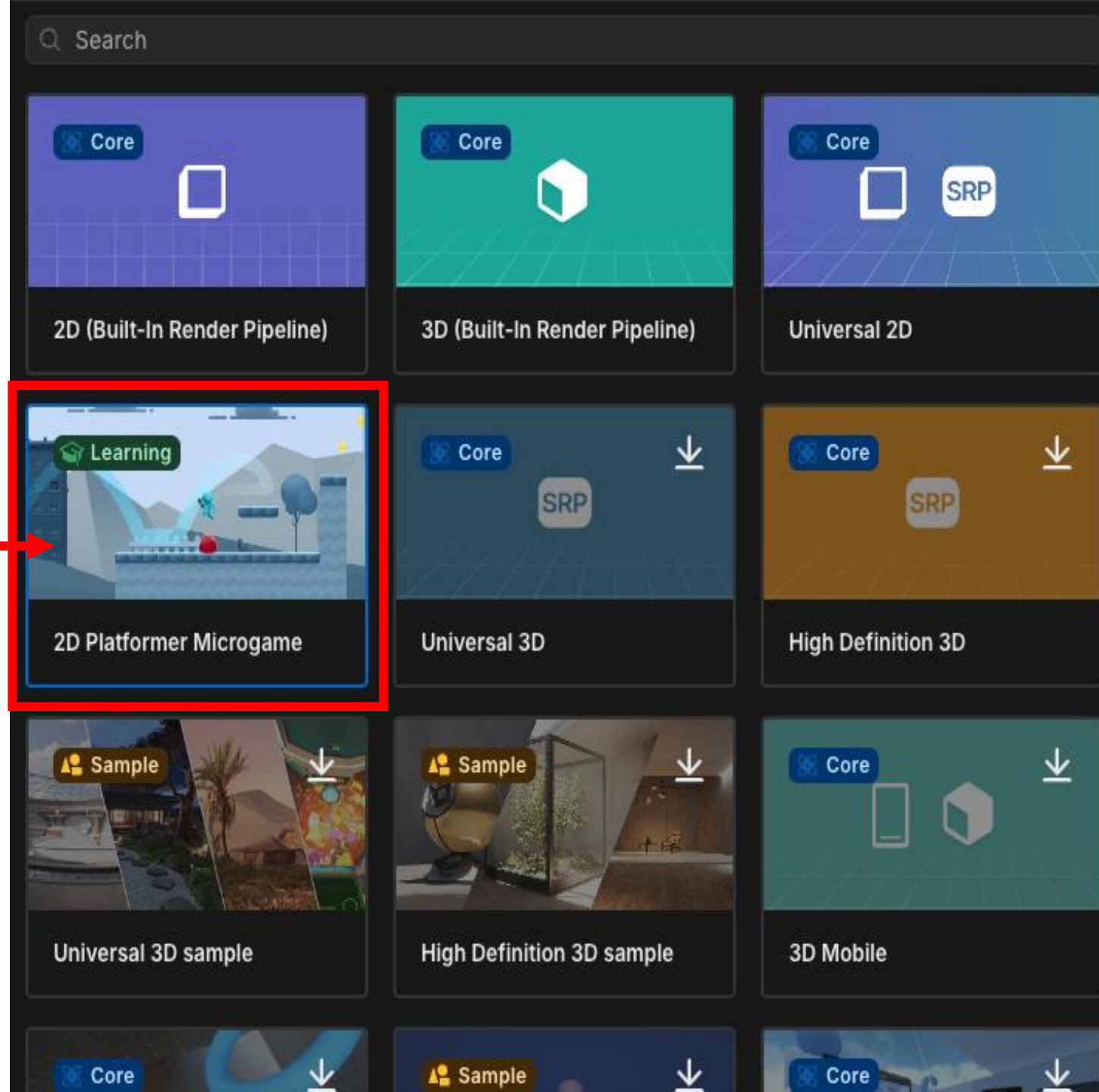


MobileControllerSystem Prefab

Drop this prefab into any scene to enable phone control + narration

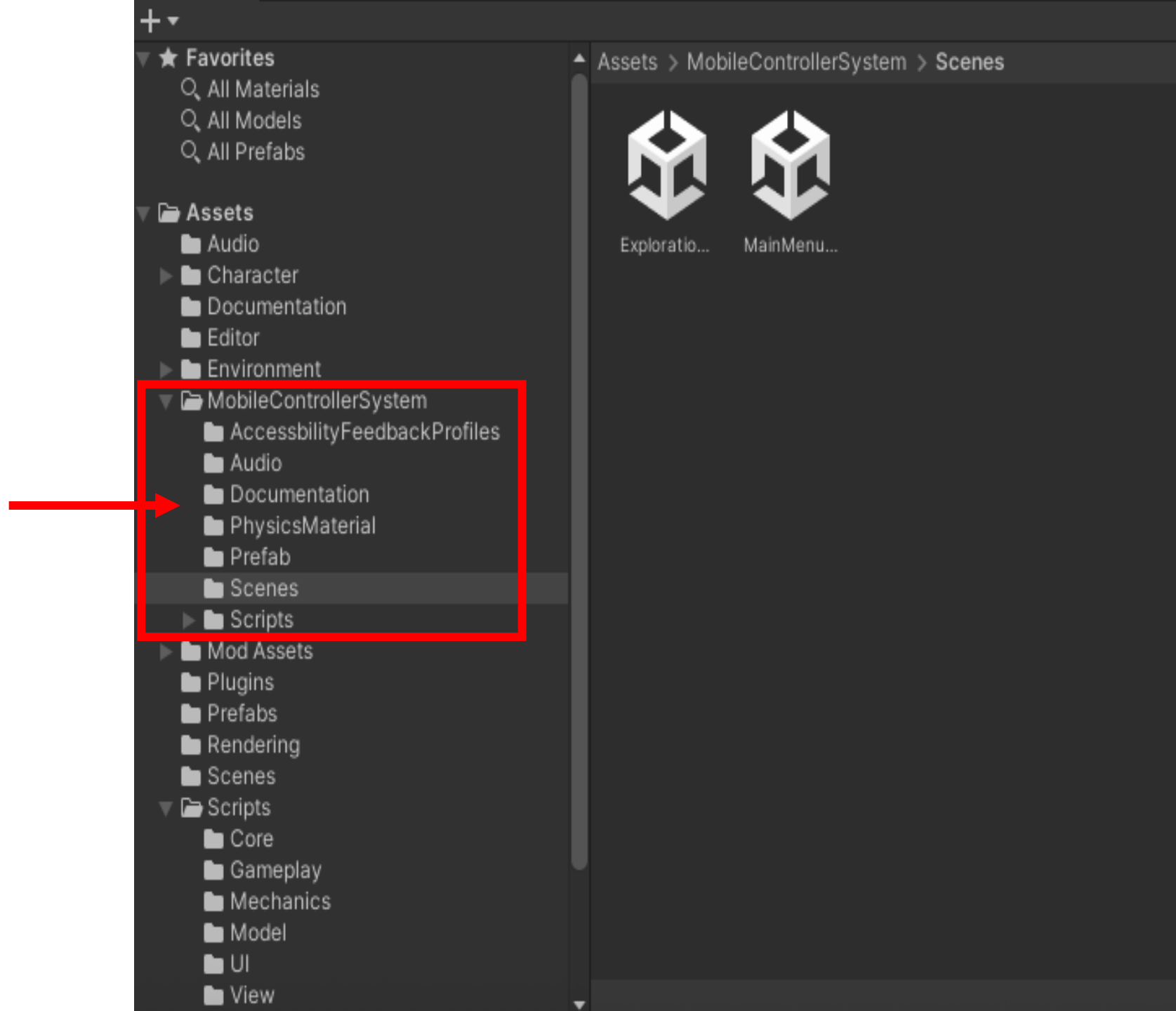
- Accessibility Events Router
- Feedback Manager
- WebSocket Connect
- Mobile Input Listener
- Game Scene Bootstrap

INTEGRATION TO A NEW GAME



Step 1

Importing the
ASCU
package



Step 2: Wiring

```
protected override void Update() {
    if (controlEnabled) {
        float inputX = 0f;
        bool jumpDown = false;
        bool jumpUp = false;
        // --- PHONE INPUT --- //
        if (usePhoneInput && phoneInput != null) {
            inputX = phoneMoveDir;
            if (phoneJumpPressed) {
                jumpDown = true;
                phoneJumpPressed = false; // consume the event
            }
        }
        // --- KEYBOARD (optional debug / fallback) ---
        if (!usePhoneInput || allowKeyboardDebug) {
            float kbX = Input.GetAxis("Horizontal");
            if (Mathf.Abs(kbX) > Mathf.Abs(inputX))
                inputX = kbX;

            if (Input.GetButtonDown("Jump"))
                jumpDown = true;
            if (Input.GetButtonUp("Jump"))
                jumpUp = true;
        }
        move.x = inputX;

        if (jumpState == JumpState.Grounded && jumpDown) {
            jumpState = JumpState.PrepareToJump;
        } else if (jumpUp) {
            stopJump = true;
            Simulation.Schedule<PlayerStopJump>().player = this;
        } else {
            move.x = 0;
        }
    }

    UpdateJumpState();
    base.Update();
}

void Awake()
{
    health = GetComponent<Health>();
    audioSource = GetComponent<AudioSource>();
    collider2d = GetComponent<Collider2D>();
    spriteRenderer = GetComponent<SpriteRenderer>();
    animator = GetComponent<Animator>();
}
```

```
protected override void Start()
{
    if (usePhoneInput && phoneInput != null)
    {
        Debug.Log("[PlayerController] Using phone input from " +
            phoneInput.name);

        // Holds for continuous movement
        phoneInput.OnHoldLeftStart += () => phoneMoveDir = -1;

        phoneInput.OnHoldLeftEnd += () => {
            if (phoneMoveDir == -1) phoneMoveDir = 0; };

        phoneInput.OnHoldRightStart += () => phoneMoveDir = 1;

        phoneInput.OnHoldRightEnd += () => {
            if (phoneMoveDir == 1) phoneMoveDir = 0; };

        // Tap / double-tap = jump
        phoneInput.OnTap += OnPhoneJump;
        phoneInput.OnDoubleTap += OnPhoneJump;
    }
}

void OnDestroy()
{
    if (phoneInput != null)
    {
        phoneInput.OnHoldLeftStart -= () => phoneMoveDir = -1;
        phoneInput.OnHoldLeftEnd -= () => {
            if (phoneMoveDir == -1) phoneMoveDir = 0; };

        phoneInput.OnHoldRightStart -= () => phoneMoveDir = 1;
        phoneInput.OnHoldRightEnd -= () => {
            if (phoneMoveDir == 1) phoneMoveDir = 0; };

        phoneInput.OnTap -= OnPhoneJump;
        phoneInput.OnDoubleTap -= OnPhoneJump;
    }
}
```


CC Unity Version Control

Hierarchy

- SampleScene*
 - GameController
 - UI Canvas
 - Main Camera
 - EventSystem
 - CM vcam1
 - CinemachineConfin
 - Player
 - Grid
 - Foreground
 - Level
 - Background
 - FarBackground
 - SpawnPoint
 - Background
 - Enemies
 - Tokens
 - Zones
 - Alien
 - MobileControllerSys

Scene

Game

Display 1

QHD (2560x1440)

Scale 0.35x

Play Focused

Stats Gizmos

Inspector

Player

Tag Player

Layer

Prefab Player

Overrides Select Open

Transform

Player Controller (Script)

Script PlayerController

Min Ground Normal Y 0.9

Gravity Modifier 1

Velocity X 0 Y 0

Jump Audio jump

Respawn Audio Death

Ouch Audio Hurt

Max Speed 3

Jump Take Off Speed 7

Jump State Grounded

Collider 2d None (Collider 2D)

Audio Source None (Audio Source)

Health None (Health)

Control Enabled

Phone Controller (optional)

Use Phone Input

Phone Input MobileControllerSystem (Mobile I

Allow Keyboard Debug

Sprite Renderer

Animator

Rigidbody 2D

Audio Source

Health (Script)

Box Collider 2D

Sprite-Unit-Default (Material)

Shader Universal Render Pipeline/2D/S Edit

Add Component

Console

Clear Collapse

18 0 0

UnityEngine.Debug:Log (object)

[16:24:42] [WebSocketTransport] Phone disconnected.

UnityEngine.Debug:Log (object)

Project

Assets > MobileControllerSystem > Scenes

Exploratio... MainMenu...

Touch Representation

Each touch gesture begins at position (x_0, y_0) at time t_0 and ends at (x_1, y_1) at time t_1 . We define displacement and duration as:

$$\begin{aligned}\Delta x &= x_1 - x_0, \\ \Delta y &= y_1 - y_0, \\ \Delta t &= t_1 - t_0\end{aligned}$$

Input Model and Gesture Mapping

The phone controller represents user input as a discrete sequence of gesture events. Let

$$G = \{ g_1, g_2, \dots, g_n \}$$

denote the sequence of gestures captured over a play session, where each gesture is represented as:

$$g = \{ a_i, t_i, m_i \}$$

where a_i is the gesture label, t_i is the gesture timestamp, and m_i is the metadata.

Input Model and Gesture Mapping

01 Menu navigation: swipes change an integer index
 $k \in \{0, \dots, K - 1\}$

02 Movement: holds are mapped to a horizontal axis
 $x_t \in [-1, 1]$ at discrete time step t .

03 Jump or Action: taps are mapped to a jump state
 $u_t \in \{0, 1\}$

Contributions

01 Drop-in MobileControllerSystem prefab for Unity

02 Phone as a controller and a separate TTS channel for gameplay gestures and UI events

03 Configurable mapping of events and feedback profiles

Limitations & Future Work

Web client constraints (iOS/browser): Package phone controller as a PWA or native app

Timing & interruption: Add device-specific TTS/gesture profiles

Connection dependency: Improve reconnect and status feedback when connection drops

Thank you!